ESCUELA DE INGENIERÍA INFORMÁTICA
PONTIFICIA UNIVERSIDAD CATÓLICA DE VALPARAÍSO

MDPI

Article

Realidad Virtual en la Industria 4.0: Prototipo funcional, ciclo de juego básico

Nico Lucy Medina [1], Anibal Palma [1]*, Francisco Espinoza [1] and Simon Ledezma [1]

[1] Pontificia Universidad Católica de Valparaíso, Valparaíso, Chile; {nico.medina.j, anibal.palma.v, francisco.espinoza.v, simon.ledezma.sc}@mail.pucv.cl
* Correspondence: anibal.palma.v@mail.pucv.cl
† Nico Lucy Medina y Anibal Palma realizaron la totalidad del desarrollo del presente informe.

Abstract: Este documento presenta el diseño inicial y la implementación preliminar de un proyecto basado en tecnología emergente. Se detallan los objetivos, la metodología utilizada y el prototipo inicial implementado como prueba de factibilidad.

Keywords: Tecnología Emergente; Industria 4.0; Realidad Virtual; Godot; Prototipo

check for updates

Received:
Accepted:
Published:

Citation: Medina, N. L.; Palma, A.; Espinoza, F.; Ledezma, S. Realidad Virtual en la Industria 4.0: Prototipo funcional, ciclo de juego básico. *Journal Not Specified* **2025**, *1*, 0.

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introducción

La realidad virtual (RV) emerge como una tecnología disruptiva que, desde hace años, ya está elevando los límites de la inmersión digital. La RV permite la creación de entornos tridimensionales (3D) simulados donde los usuarios pueden interactuar de manera natural con gestos en la realidad, traducidos en eventos virtuales, ofreciendo una sensación de inmersión superior a solo estar frente a un ordenador [1].

En la última década, la RV ha evolucionado desde prototipos experimentales hasta convertirse en una plataforma cada vez más común para experiencias de ocio. Este crecimiento ha sido impulsado por dispositivos más accesibles como Meta Quest 2, que han democratizado el acceso a experiencias de RV de alta calidad.

El proyecto *Realidad Virtual en la Industria 4.0: Prototipo funcional, ciclo de juego básico* se enmarca en este contexto, explorando las posibilidades de la RV como plataforma para experiencias lúdicas inmersivas. A través de un prototipo funcional desarrollado en Godot Engine, buscamos demostrar:

- La viabilidad técnica de implementar mecánicas básicas de juego en RV.
- La efectividad de los sistemas de interacción natural mediante seguimiento de manos.
- El potencial de los entornos virtuales para crear experiencias envolventes.

Este trabajo representa tanto una demostración técnica como una exploración creativa de las posibilidades que ofrece la RV en el ámbito del entretenimiento interactivo, sentando las bases de un prototipo para desarrollos más ambiciosos en el futuro.

Desarrollo técnico

- **Motor gráfico:** Godot Engine (versión 4.4), seleccionado por su flexibilidad y soporte nativo para RV.
- **Hardware:** Meta Quest 2, aprovechando sus capacidades de seguimiento de manos y cabeza, gatillos físicos, botones de interacción y renderizado side-by-side para la visualización estereoscópica (se considera a los mandos como parte del dispositivo).

- **Interacción básica:**
  - Mecánica principal: Disparo de proyectiles mediante gestos manuales.
  - Objetivo: Derrota de enemigos en un entorno virtual simplificado.

Alcance y perspectivas

La demo actual funciona como *proof of concept*, centrándose en los elementos esenciales para una experiencia de RV inmersiva, pero con un diseño escalable que permitiría su expansión hacia un juego completo con mecánicas más complejas (como interacciones multitáctiles, inteligencia artificial avanzada o integración con sistemas industriales simulados o cualquier propósito que sea conveniente).

2. Objetivos del Proyecto

2.1. Objetivo General

Desarrollar un programa "Hola Mundo" o "Prueba de Concepto" que demuestre la viabilidad de implementar funcionalidades esenciales de realidad virtual en un entorno de simulación de juego, utilizando Godot Engine 4.4 con GDScript. La demostración se centrará en aspectos técnicos como la visualización estereoscópica, la integración del *hand tracking* y la interacción básica (disparo) en un entorno simulado.

2.2. Objetivos Específicos

- **Implementar visualización estereoscópica:** Configurar y optimizar la cámara para que ofrezca una representación *side-by-side* adecuada para visores de RV, garantizando una experiencia inmersiva.
- **Integrar *hand tracking*:** Incorporar el seguimiento de manos que permita interpretar y reflejar los movimientos del usuario, facilitando interacciones naturales y precisas.
- **Desarrollar mecánica básica de disparo:** Permitir la interacción a través de controles (ej. gatillo o botón) para disparar proyectiles, implementando la lógica básica de interacción.
- **Gestionar la aparición aleatoria de enemigos:** Programar la aparición de enemigos en posiciones aleatorias para que la mecánica de disparo y la interacción del usuario sean probadas en condiciones diversas.
- **Optimizar el rendimiento:** Garantizar que la aplicación funcione fluidamente en dispositivos de RV compatibles (como Meta Quest 2), minimizando la latencia en el seguimiento de manos y manteniendo la estabilidad durante la demostración.

3. Metodología de Desarrollo

Dada la naturaleza de esta prueba de concepto, con un alcance claramente definido y enfocado en la validación de funcionalidades técnicas clave, se propuso la utilización de la metodología en cascada. Esta elección se justifica por los siguientes motivos:

- **Alcance delimitado y etapas bien definidas:** La demostración técnica requiere la definición clara de requisitos y la verificación de funcionalidades puntuales, lo que se ajusta a la secuencia lineal de la metodología en cascada.
- **Facilidad en la planificación:** El desarrollo se estructura en fases (investigación, diseño, implementación) con hitos específicos, permitiendo una planificación precisa, particularmente valiosa en equipos pequeños.
- **Validación progresiva:** Cada fase finalizada (por ejemplo, el diseño de requisitos y el modelado del sistema) se revisa antes de iniciar la siguiente etapa, reduciendo el riesgo de desviaciones en funcionalidades críticas.

4. Plan y Programa de Trabajo: Estructura de Desglose del Trabajo (EDT)

4.1. Fase 1: Investigación

4.1.1. Estado del arte de la realidad virtual en Godot

Godot Engine ha avanzado significativamente en su soporte para realidad virtual (RV), consolidándose como una opción viable para desarrolladores independientes y pequeños estudios. A partir de la versión 4, Godot incorpora soporte nativo para OpenXR, lo que permite compatibilidad directa con dispositivos como Meta Quest 2 y 3 [2]. El motor cuenta con nodos específicos para realidad virtual —la unidad básica de los objetos en el motor— de tipo XR (como `XRController3D` y `XROrigin`), y un conjunto de utilidades agrupadas en el paquete *Godot XR Tools*, que incluye sistemas listos para navegación, interacción con objetos e incluso *assets* prefabricados [3].

Recientemente, se han presentado avances clave: Meta ha financiado el desarrollo de mejoras en la integración de OpenXR y en el soporte para Quest, y ha lanzado una versión de Godot que puede ejecutarse directamente en dispositivos Meta Quest 2 en adelante [4]. Esto permite una experiencia de desarrollo más fluida. Asimismo, en la *Asset Library* oficial de Godot se encuentran ejemplos de implementación de seguimiento corporal y de manos [5], lo que reafirma al motor como un entorno apto para el desarrollo de experiencias interactivas de RV.

En cuanto a documentación, Godot provee guías claras y actualizadas sobre todos los elementos del motor, incluyendo una extensa sección dedicada a la configuración del entorno XR, sus métodos y variables [2]. Esta accesibilidad se ve complementada por una comunidad activa, que produce tutoriales, complementos y plantillas reutilizables, reforzando así el ecosistema de RV del motor.

4.1.2. Análisis de tecnologías disponibles

Curva de aprendizaje

- **Godot:** Su lenguaje propio, GDScript, facilita una curva de aprendizaje suave, especialmente para nuevos desarrolladores o aquellos enfocados en prototipos rápidos, debido a su similitud con *Python*, tanto en la sintaxis como en el uso de *keywords*, y al tratarse de un lenguaje interpretado. Su interfaz es clara, ligera y con un flujo de trabajo sencillo. Para quienes ya están inmersos en el mundo de la programación, Godot ofrece integración con el lenguaje C++ para quienes así lo deseen [2].
- **Unity:** Utiliza C#, un lenguaje ampliamente conocido, con una curva de aprendizaje moderada. Cuenta con abundantes recursos, pero su entorno es más pesado que el de Godot [6].
- **Unreal Engine:** Utiliza C++ y *Blueprints* (programación visual), con una curva de aprendizaje más empinada. Requiere una gran cantidad de *Blueprints* para lograr resultados aceptables. Ofrece herramientas de alto nivel, pero requiere más tiempo de aprendizaje, y su rendimiento no siempre es el mejor [7].

Requisitos de hardware

- **Godot:** Ligero y optimizado. Funciona bien en equipos modestos (i5, GPU de gama media como la GTX 1050). Ideal para desarrolladores con recursos limitados [2].
- **Unity:** Requiere hardware de gama media-alta. Funciona fluidamente en PCs modernos, pero puede demandar más recursos en proyectos de RV [6].
- **Unreal Engine:** Tiene una alta exigencia de recursos. Necesita GPU y CPU potentes para desarrollos en RV con gráficos avanzados [7].

Soporte para RV (especialmente Meta Quest 2)

- **Godot:** Soporte nativo para OpenXR. Incluye simulador de RV y herramientas como XR Tools. Compatible con seguimiento de manos y cuerpo [3].
- **Unity:** Soporte maduro mediante XR Plugin Management y Oculus SDK. Cuenta con abundante documentación y una comunidad especializada en RV [6].
- **Unreal Engine:** Ofrece plantillas de RV integradas, compatibilidad con OpenXR y Oculus SDK. Es ideal para experiencias de alta fidelidad visual [7].

Licenciamiento

- **Godot:** Gratuito y *open source* bajo licencia MIT [2].
- **Unity:** Modelo *freemium*, con restricciones según los ingresos y licencias pagas. Ha generado controversias por sus constantes cambios en los términos y condiciones, lo que evidencia un modelo de negocio impredecible y poco amigable para desarrolladores independientes [8].
- **Unreal Engine:** Gratuito, pero con pago de *royalties* después del primer millón de dólares (USD) en ingresos [7].

4.1.3. Justificación del uso de Godot para esta implementación

En general, Godot es una buena opción por su sistema de manejo de nodos, similares a los *Blueprints* de Unreal, pero con una mayor flexibilidad. Para este prototipo en particular, representa una alternativa estratégica para el desarrollo de una demostración técnica de realidad virtual interactiva en un entorno de videojuego. Su soporte nativo para OpenXR, junto con herramientas específicas como Godot XR Tools y la existencia de plantillas y ejemplos funcionales de seguimiento de manos y cuerpo, permite implementar interacciones en RV con relativa facilidad [3].

El motor es altamente accesible, tanto en términos económicos (gratuito y *open source*) como técnicos. Su lenguaje GDScript reduce la complejidad de la programación y acorta los ciclos de desarrollo, facilitando la iteración y el prototipado ágil [2]. Estas características lo convierten en una herramienta idónea para equipos pequeños que desean explorar y validar experiencias en RV sin comprometer recursos excesivos.

Finalmente, la comunidad activa y colaborativa de Godot, junto con su documentación clara y en constante actualización, proporciona un ecosistema de soporte robusto [9]. Esto asegura que los desarrolladores cuenten con los recursos necesarios para resolver problemas, integrar nuevas funcionalidades y mantener una curva de aprendizaje sostenida.

Por tanto, la elección de Godot para esta demostración técnica no solo es viable, sino que está respaldada por su evolución tecnológica, su modelo abierto y su orientación práctica hacia desarrolladores independientes y de bajo presupuesto.

4.2. Fase 2: Diseño

4.2.1. Definición de Requisitos

Requisitos Funcionales

Los requisitos funcionales definen el comportamiento esperado del sistema frente a las acciones del usuario o eventos del entorno virtual. En este proyecto, se han clasificado en función de las capacidades clave del prototipo de realidad virtual:

- **Visualización Estereoscópica:** El sistema debe proporcionar una representación visual en estéreo del entorno simulado, utilizando renderizado dual por cámara para cada ojo de forma sincronizada. Esta funcionalidad es esencial para lograr una experiencia inmersiva y generar la sensación de profundidad al usuario propia de entornos de realidad virtual. Esta debe ser totalmente compatible con dispositivos Meta Quest 2.

- **Seguimiento de Manos (Hand Tracking):** El sistema debe detectar y reflejar en tiempo real la posición, orientación y gestos de las manos del usuario, permitiendo una interacción natural con los objetos del entorno sin necesidad de utilizar controladores físicos. Para esto se simulará un símil virtual de la mano del usuario mediante mallas 3D y un sistema simple de colisiones.
- **Interacción con Objetos Físicos:** El sistema debe permitir que el jugador agarre y suelte objetos definidos como cuerpos rígidos (`RigidBody3D`). Si el objeto es un arma, el agarre debe realizarse a través de un punto específico denominado `AttachmentPoint`; si es un objeto genérico, debe ser tomado por su centro físico.
- **Mecánica de Disparo:** El sistema debe permitir al jugador disparar proyectiles presionando botones, en este caso el botón que más sea natural, siendo este el gatillo derecho del dispositivo Meta Quest 2. Estos proyectiles deberán seguir una trayectoria física y colisionar correctamente con enemigos u otros elementos interactivos del entorno y generar una respuesta acorde.
- **Aparición Aleatoria de Enemigos:** El sistema debe generar enemigos en posiciones aleatorias dentro del entorno virtual a intervalos regulares definidos (cada 2 segundos) y de esta forma generar un ciclo de juego simple donde aparecen enemigos y el jugador debe destruirlos.
- **Movimiento del Jugador:** El jugador debe poder desplazarse por el entorno mediante *joystick*, desplazando todo el personaje mediante el uso de `CharacterBody3D`. Además, debe tener la capacidad de agacharse físicamente o a través de una entrada digital usando un botón del mando (botón Y/B).
- **Sistema de Combate:** Los enemigos deben poseer un atributo de vida (`HP`) de tipo flotante. Al recibir daño, dicho valor debe disminuir. Cuando el `HP` llegue a cero, el enemigo debe ejecutar la función `queue_free()` y ser eliminado del entorno de juego. Adicionalmente, el jugador poseerá un atributo llamado "LVL" o "Level" para representar la progresión conforme avanza el ciclo de juego; cada enemigo otorga "EXP" al jugador. Al superar un máximo, el jugador aumenta su LVL.

Requisitos No Funcionales

Los requisitos no funcionales definen las condiciones bajo las cuales el sistema debe operar, estableciendo criterios de calidad, rendimiento, usabilidad y compatibilidad que no están directamente relacionados con funcionalidades específicas.

- **Rendimiento del Sistema (Plataforma):** El sistema debe mantener una tasa de refresco estable de al menos 72 FPS y la resolución máxima (1832 x 1920 por ojo) durante su ejecución en dispositivos Meta Quest 2, para asegurar una experiencia fluida. Se seguirán estándares de mallas 3D de polígonos reducidos para preservar el rendimiento a costa de menor calidad de objetos.
- **Tiempo de Respuesta:** Las acciones realizadas por el usuario (como disparar, agarrar objetos o moverse) deben reflejarse en pantalla en un tiempo inferior a 10 ms desde el momento del *input*.
- **Portabilidad:** El sistema debe ser ejecutable desde un computador, conectado de manera alámbrica/inalámbrica al dispositivo Meta Quest 2.
- **Usabilidad e Interacción Natural:** La interfaz de usuario debe estar diseñada para permitir una interacción intuitiva sin necesidad de entrenamiento previo. Los gestos de mano, las acciones de disparo y los movimientos deben ser naturales y de fácil comprensión.
- **Robustez ante Entradas Inválidas:** El sistema debe ser capaz de ignorar interacciones del usuario con objetos no diseñados para ser manipulables, sin generar errores ni afectar el rendimiento general.

- **Tolerancia a Fallas en el Tracking:** En caso de pérdida del seguimiento de manos o del movimiento del jugador, el sistema debe mantener el último estado válido y proporcionar retroalimentación visual para evitar confusión.
- **Accesibilidad:** El sistema debe ofrecer modos alternativos de locomoción (como movimiento alternativo con la palanca del controlador para ajustar posición y rotación) que se adapten a las preferencias o necesidades específicas de cada usuario.

4.2.2. Modelado del sistema

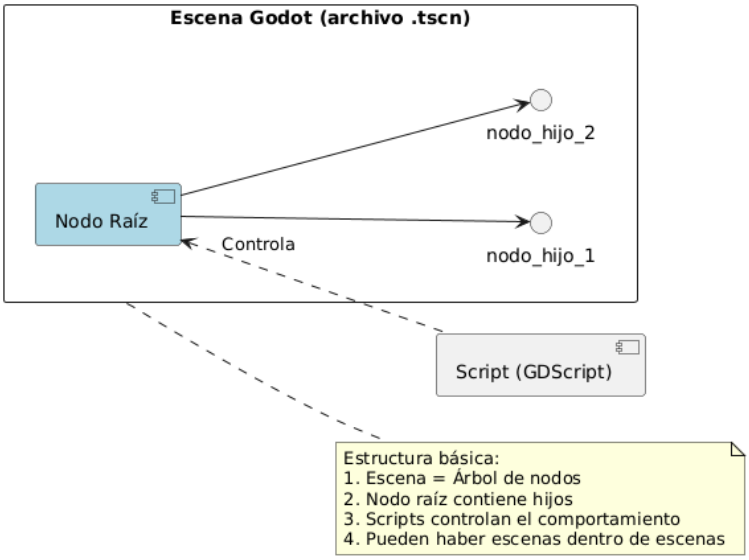

Figura 1. Estructura de escenas y nodos del sistema.

Explicación de la Figura 1: Este diagrama modela la estructura jerárquica de escenas y nodos en Godot Engine, clave para entender cómo se organiza el prototipo del juego en RV. A continuación, se detallan sus componentes y su relación con el proyecto:

1. **Escena Godot (archivo `.tscn`):** Representa una escena en Godot, guardada como archivo `.tscn` (formato de texto para escenas).
   - **Nodo raíz (color azul claro):** Es el nodo padre principal de la escena (p. ej., `Node3D` o `XROrigin3D` en el prototipo).
   - **Hijos directos:** `nodo_hijo_1` y `nodo_hijo_2` (como `XRCamera3D`, `Player`, o `EnemySpawner` en el proyecto).
   - **Anidación de escenas:** Godot permite incluir escenas dentro de otras (ejemplo: `Player.tscn` instanciada en `MainScene.tscn`).
2. **Script (GDScript):** Controla el comportamiento del nodo raíz y sus hijos.
   Ejemplo de script en Godot (MainScene.gd):

   ```
   extends Node3D

   func _ready():
       var jugador = preload("res://Player.tscn").instantiate()
       add_child(jugador)
   ```

   - **Relación con el diagrama:** La flecha punteada indica que el script se asocia al nodo raíz para gestionar lógica (ejemplo: spawn de enemigos o interacción con RV).
3. **Nota explicativa:** Detalla cuatro principios clave de Godot:

- **Estructura arbórea:** Las escenas son árboles de nodos (ejemplo: `MainScene` contiene `XROrigin3D`, que a su vez contiene `XRCamera3D` y `XRController3D`).
- **Control por scripts:** Los nodos son inertes sin scripts; estos añaden funcionalidad (ejemplo: `HandTracking.gd` en `XRController3D`).
- **Escenas anidadas:** Permiten modularidad (ejemplo: `Enemy.tscn` instanciada por `EnemySpawner.gd`).
- **Paquete Completo:** Permite desarrollar todo tipos de juegos, contando con dos motores principales, 2D y 3D. permitiendo generar entornos de dos dimensiones y luego ejecutarlos en 3 dimensiones.

4.3. Fase 3: Planificación

4.3.1. Programa de Actividades y RRHH

Inicialmente, el equipo proyectado constaba de cuatro integrantes. Sin embargo, durante la ejecución del proyecto, el equipo efectivo se redujo a dos miembros activos: Nico Lucy Medina y Anibal Palma. Los otros dos integrantes originalmente considerados se ausentaron y no participaron en el desarrollo, sin establecer comunicación con el equipo en ningún momento, ni siquiera durante el último día de entrega.

La asignación de tareas se adaptó a esta nueva estructura, siguiendo la planificación general esbozada en la Carta Gantt de la Figura 2:

Nico Lucy Medina: Asumió la responsabilidad principal de la Fase 1: Investigación y Diseño. Esto incluyó tareas como la revisión bibliográfica, la definición de requisitos funcionales y no funcionales, y el modelado del sistema (incluyendo diagramas de casos de uso, secuencia, UML y estados). Adicionalmente, colaboró en la fase final de consolidación, preparación del informe y material audiovisual.

Anibal Palma: Se centró en la Fase 2: Implementación y Pruebas. Fue responsable del desarrollo técnico del prototipo "Hola-Mundo" en Godot Engine, abarcando la implementación de la visualización estereoscópica, la integración del seguimiento de manos (hand tracking), el desarrollo de la mecánica de disparo, la programación de la aparición de enemigos, la implementación del movimiento del jugador y el sistema básico de combate. También se encargó de la integración de estas funcionalidades y de las pruebas preliminares. Además, mantuvo una comunicación constante en clases con el profesor a cargo de supervisar el proyecto.

Esta distribución permitió cubrir las etapas críticas del proyecto, desde la conceptualización y diseño hasta la implementación técnica y validación del prototipo.

4.3.2. Carta Gantt

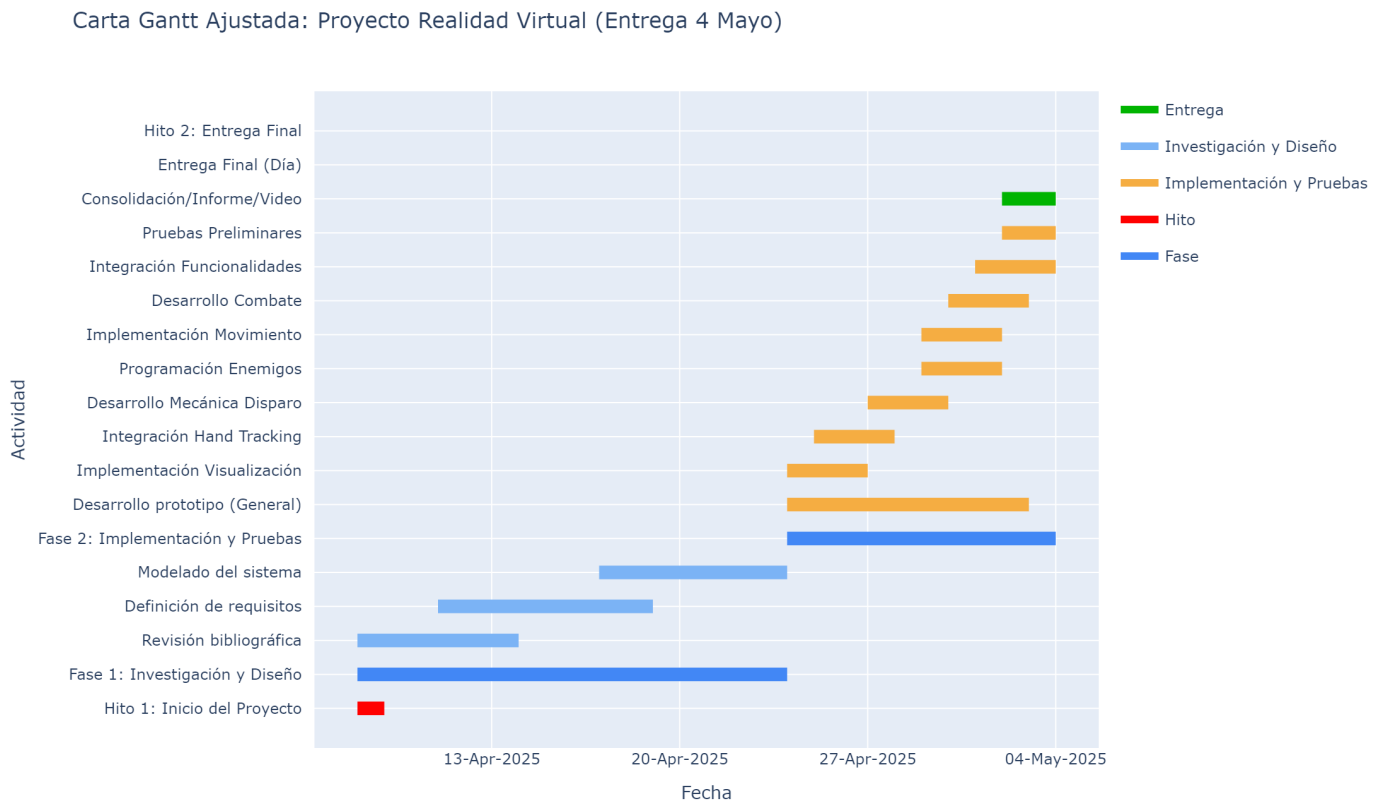

Figura 2. Carta Gantt del proyecto, mostrando la planificación temporal ajustada de las actividades.

La Figura 2 muestra la planificación temporal ajustada retroactivamente del proyecto, organizada por hitos, fases y actividades específicas, para reflejar una fecha de término el 4 de mayo de 2025:

- **Hito 1: Inicio del Proyecto**
  - Inicio: 8 de abril de 2025
  - Fin: 9 de abril de 2025
- **Fase 1: Investigación y Diseño** (8 al 24 de abril de 2025)
  - Revisión bibliográfica: 8–14 de abril
  - Definición de requisitos: 11–19 de abril
  - Modelado del sistema: 17–24 de abril
- **Fase 2: Implementación y Pruebas** (24 de abril al 4 de mayo de 2025)
  - Desarrollo del prototipo (general): 24 de abril – 3 de mayo
    - Implementación Visualización: 24–27 de abril
    - Integración Hand Tracking: 25–28 de abril
    - Desarrollo Mecánica de Disparo: 27–30 de abril
    - Programación Enemigos: 29 de abril – 2 de mayo
    - Implementación Movimiento: 29 de abril – 2 de mayo
    - Desarrollo Combate: 30 de abril – 3 de mayo
  - Integración de funcionalidades: 1–4 de mayo
  - Pruebas preliminares: 2–4 de mayo

- **Consolidación / Informe / Video** (2–4 de mayo de 2025)
  - Preparación de Entregables: 2–4 de mayo
- **Hito 2: Entrega Final**
  - Inicio: 4 de mayo de 2025
  - Fin: 4 de mayo de 2025
  - Entrega final: 4 de mayo

5. Diseño del Prototipo

5.1. Requisitos Funcionales y No Funcionales

Cuadro 1. Requerimientos funcionales y no funcionales del prototipo de realidad virtual.

Tipo	Subtipo	Especificación	ID	Descripción y criterios de aceptación
Funcional	ALGORITMOS	Visualización	RF-01	Visualización estereoscópica mediante renderizado dual por cámara (side-by-side); debe ser compatible con Meta Quest 2.
Funcional	ALGORITMOS	Seguimiento	RF-02	Seguimiento de manos con latencia menor a 50 ms; gestos del usuario reflejados en tiempo real mediante malla 3D.
Funcional	ALGORITMOS	Mecánica	RF-03	Disparo mediante gatillo derecho; proyectiles como `RigidBody3D` con trayectorias físicas y colisión.
Funcional	ALGORITMOS	Generación	RF-04	Enemigos aparecen cada 2 segundos en ubicaciones aleatorias del entorno, sin colisión inicial.
Funcional	FUNCIONES ADMINISTRATIVAS	Registro	RF-05	Registro opcional de eventos de disparo y errores con timestamp.
Funcional	ALGORITMOS	Movimiento	RF-06	Agacharse de forma física o usando botones Y/B; la cámara ajusta la altura del jugador en ambos casos.
Funcional	ALGORITMOS	Movimiento	RF-07	Locomoción mediante joystick utilizando `CharacterBody3D`; desplazamiento libre en el entorno.
Funcional	ALGORITMOS	Interacción	RF-08	Agarre y suelta de objetos `RigidBody3D` mediante gestos naturales del usuario.
Funcional	ALGORITMOS	Interacción	RF-09	Agarre contextual: armas deben tomarse desde el nodo `AttachmentPoint`; otros objetos desde el centro físico.
Funcional	ALGORITMOS	Diseño	RF-10	Todas las armas deben tener definido un nodo `AttachmentPoint` para permitir interacción adecuada.
Funcional	INFORMACIÓN Y DATOS	Datos	RF-11	Los enemigos deben tener un atributo HP de tipo `float` que se reduce al recibir impactos.
Funcional	ALGORITMOS	Mecánica	RF-12	Si el HP del enemigo es menor o igual a 0, debe ejecutarse `queue_free()` para eliminarlo del entorno.
No funcional	RENDIMIENTO: PLATAFORMA	Rendimiento	RNF-01	El sistema debe mantener 72 FPS estables y resolución de 1832x1920 por ojo en Meta Quest 2.
No funcional	RENDIMIENTO: RESPUESTA	Latencia	RNF-02	Las acciones del usuario deben reflejarse en menos de 10 ms desde el input hasta la visualización.
No funcional	PORTABILIDAD	Compatibilidad	RNF-03	Compatible con ejecución nativa en Quest 2 y desde PC vía conexión alámbrica o inalámbrica.
No funcional	USABILIDAD	Interfaz	RNF-04	Interfaz e interacciones intuitivas que no requieran tutorial; gestos deben sentirse naturales.
No funcional	EFICIENCIA	Optimización	RNF-05	Uso de modelos low-poly, shaders simples y técnicas de oclusión activa para optimizar el rendimiento.
No funcional	ROBUSTEZ	Estabilidad	RNF-06	El sistema debe ignorar intentos de interacción inválidos sin generar errores ni comprometer la estabilidad.
No funcional	TOLERANCIA A FALLAS	Recuperación	RNF-07	Ante pérdida de tracking, debe mantener el último estado válido y ofrecer retroalimentación visual o háptica.
No funcional	ACCESIBILIDAD	Adaptabilidad	RNF-08	Locomoción alterna (joystick o teletransporte) debe ser configurable según las preferencias del usuario.

5.2. Diagramas

5.2.1. Diagrama de Casos de Uso

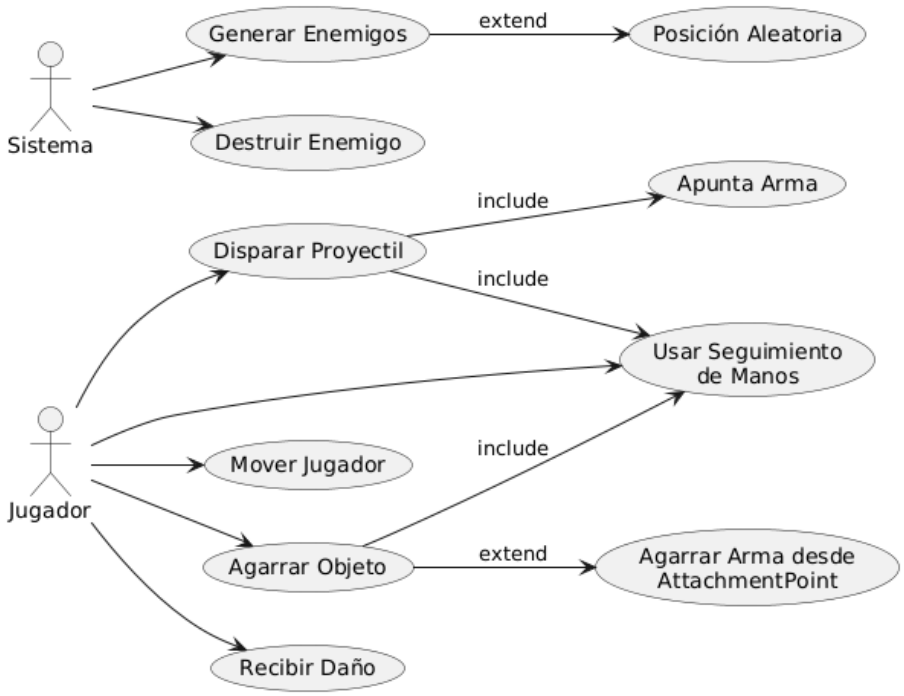

Figura 3. Diagrama de casos de uso.

Este diagrama (Figura 3) modela las interacciones entre el Jugador y el sistema, identificando los casos de uso clave:

- Actor principal: Jugador (opera el visor Meta Quest 2 y un mando por cada mano).
- Actor secundario: Sistema (procesos automáticos como generación de enemigos).
- Casos de uso destacados:
  - **«include»**:
    - Disparar Proyectil requiere Apunta Arma.
    - Agarrar Objeto y Disparar Proyectil dependen de Usar Seguimiento de Manos.
  - **«extend»**:
    - Generar Enemigos se extiende con Posición Aleatoria.
    - Agarrar Objeto se especializa en Agarrar Arma desde AttachmentPoint.

5.2.2. Mockup de Vista Principal

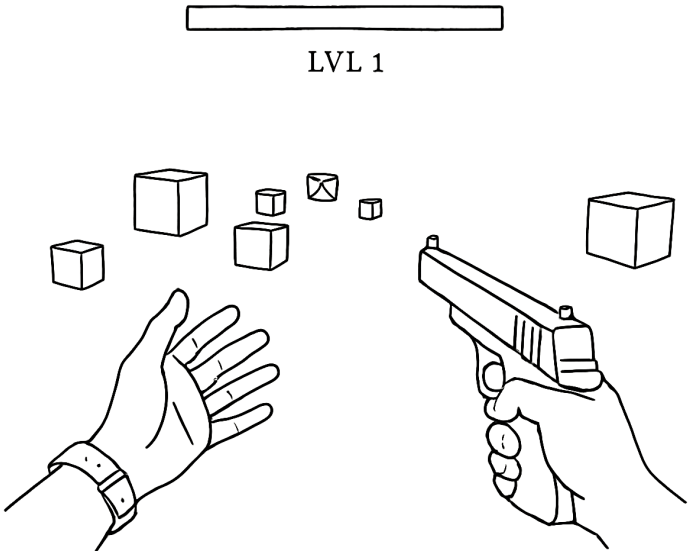

Figura 4. Mockup de la vista principal en el entorno de RV.

Este mockup (Figura 4) representa la interfaz visual que el jugador observa dentro del entorno de realidad virtual. Se destacan los elementos clave del HUD, la posición relativa de las manos, armas y enemigos, proporcionando una referencia visual temprana del diseño interactivo y estético del prototipo.

5.2.3. Diagramas de Secuencia
a. Secuencia de Disparo

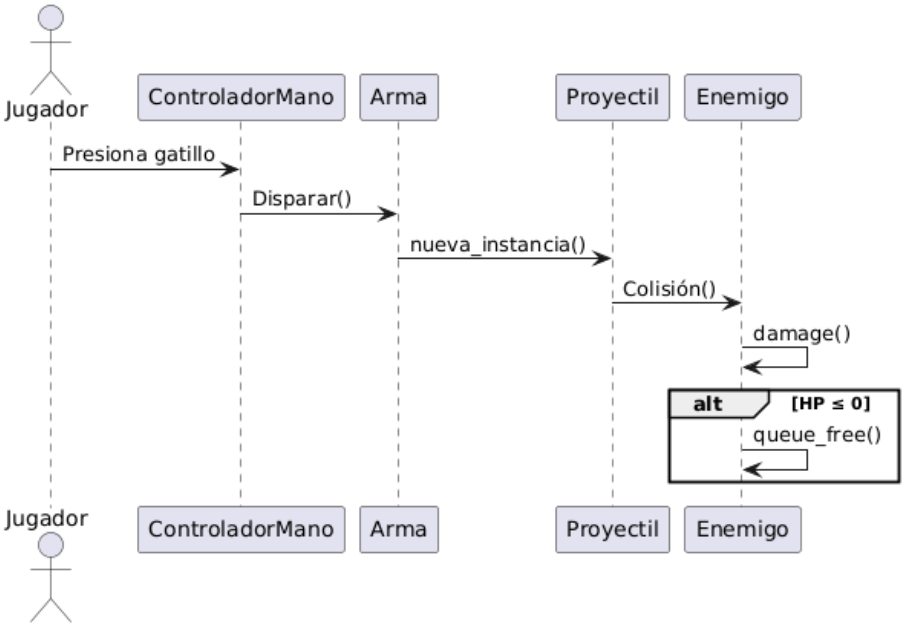

Figura 5. Diagrama de secuencia: Disparo.

La Figura 5 modela el flujo de disparo de un proyectil:

- Jugador presiona el gatillo (detectado por `ControladorMano`).
- Arma instancia un `Proyectil` (clase `RigidBody3D`).

- Proyectil impacta en `Enemigo`, reduciendo su HP (`reduceHP()`).
- Si HP ≤ 0, el enemigo se elimina (`queue_free()`).

b. Secuencia de Generación de Enemigos

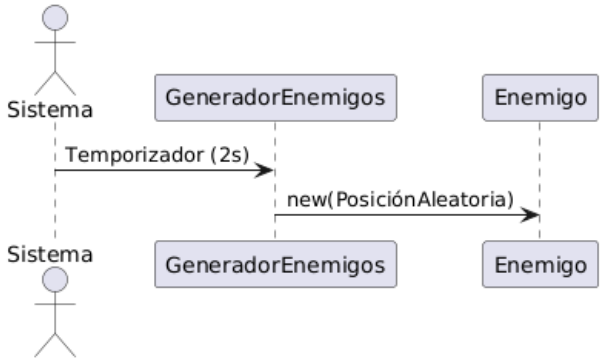

Figura 6. Diagrama de secuencia: Enemigo.

La Figura 6 muestra cómo el Sistema genera enemigos cada 2 segundos:

- `GeneradorEnemigos` recibe señal del temporizador.
- Instancia un `Enemigo` en posición aleatoria dentro del radio definido.

5.2.4. Diagrama UML (Vista Estática)

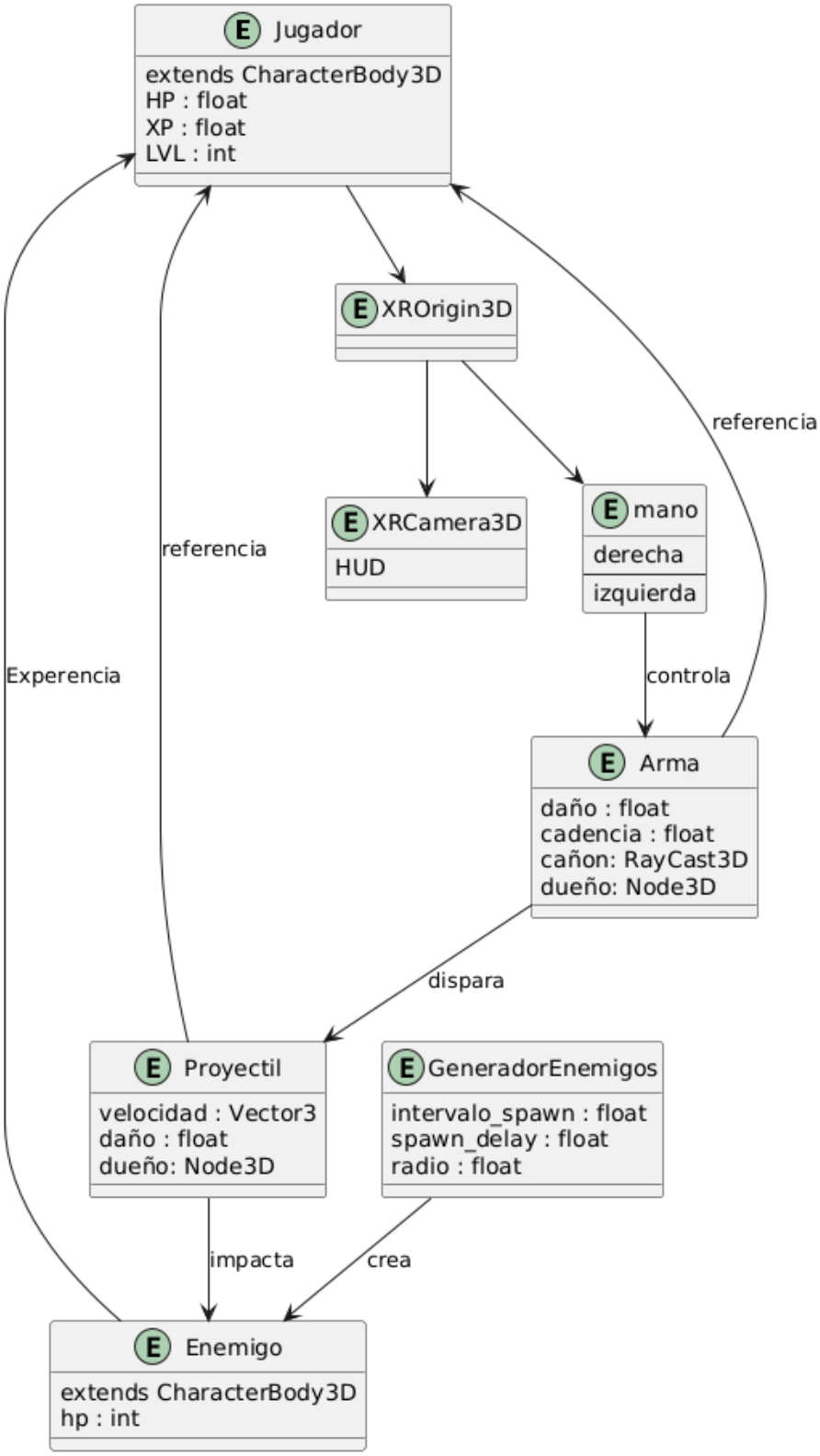

Figura 7. Diagrama UML.

La Figura 7 representa la estructura de clases del sistema:

- Clases clave:
  - **Jugador**: Hereda de `CharacterBody3D` (movimiento y física).

- **Arma**: Contiene atributos como daño y cadencia, vinculada al `XROrigin3D`.
- **Enemigo**: Con HP y lógica de destrucción.
- **GeneradorEnemigos**: Controla intervalos de `spawn` (`spawn_delay`).

- Relaciones:
  - Agregación: `GeneradorEnemigos` crea múltiples `Enemigo`.
  - Asociación: `Arma` dispara `Proyectil`, que impacta en `Enemigo`.

5.2.5. Diagramas de Estados (Vista Dinámica)
a. Estado de Generación de Enemigos

Figura 8. Diagrama de estados: Generación de enemigos.

La Figura 8 modela el ciclo de vida de un enemigo:

- **Generado**: Instanciado en escena.
- **Vivo**: Ejecuta comportamientos (moverse, atacar).
- **Muerto**: Eliminado al recibir daño crítico (HP ≤ 0).

b. Estado del Jugador

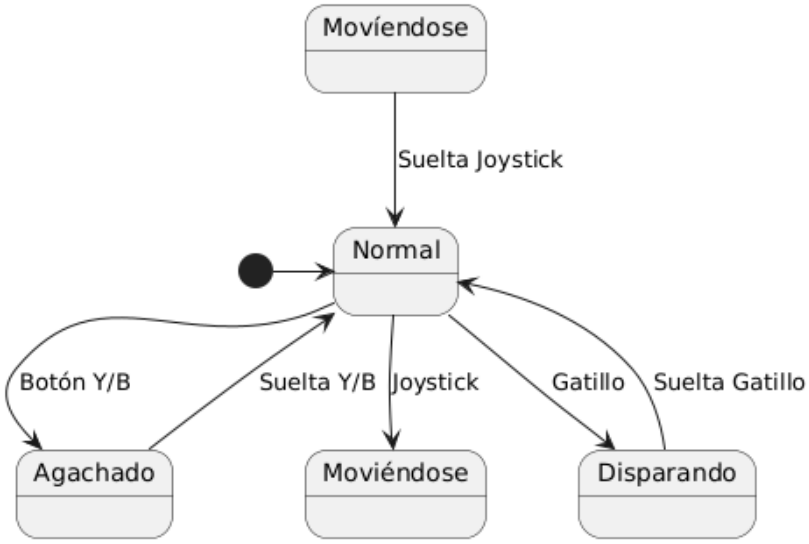

Figura 9. Diagrama de estados: Jugador.

La Figura 9 detalla los estados del jugador:

- **Normal**: Estado base.
- **Agachado**: Activado por botones Y/B (ajuste de altura de cámara).
- **Moviéndose**: Desplazamiento con joystick (`CharacterBody3D.move_and_slide()`).
- **Disparando**: Gatillo presionado (bloquea otras acciones).

6. Implementación del Hola-Mundo

La implementación del prototipo se realizó utilizando Godot Engine versión 4.4, seleccionado por su flexibilidad, soporte nativo para RV a través de OpenXR y su licencia de código abierto. El desarrollo se enfocó en validar las funcionalidades más esenciales para una experiencia de RV básica en un dispositivo Meta Quest 2.

El proceso general siguió estos pasos clave, basados en los requisitos y el diseño definidos:

- **Configuración del Entorno XR:** Se estableció la estructura básica de la escena utilizando nodos específicos de XR en Godot, como `XROrigin3D` y `XRCamera3D`, para manejar el origen del espacio de seguimiento o "trackingz la cámara estereoscópica que renderiza la vista para cada ojo (side-by-side).
- **Integración del Seguimiento de Manos (Hand Tracking):** Se implementó el seguimiento de las manos del usuario utilizando las capacidades nativas de Godot y OpenXR. Se añadieron representaciones visuales de las manos (mallas 3D), las cuales se deforman para representar el apretar un botón en el mundo real, además se agregaron áreas de colisión para permitir la interacción básica con el entorno.
- **Navegación con controles tradicionales:** Se habilitó el desplazamiento y rotación de la cámara del jugador por el entorno usando el joystick (derecho) del controlador, gestionado a través de un nodo `CharacterBody3D`. También se implementó la capacidad de agacharse físicamente o mediante un botón (B/Y).
- **Agarre de Objetos:** Se programó la funcionalidad para agarrar y soltar objetos `RigidBody3D`, diferenciando el punto de agarre para armas (`AttachmentPoint`) y objetos genéricos (centro de masa).
- **Disparo:** Se implementó la mecánica de disparo activada por el gatillo del controlador. Los proyectiles se instanciaron como `RigidBody3D` para que siguieran trayectorias físicas y pudieran colisionar con otros elementos.
- **Implementación del Ciclo de Juego Básico:** Se creó un sistema simple para la aparición (spawn) de enemigos. Estos enemigos aparecen en posiciones aleatorias (mediante la función `randi()`) y pueden ser eliminados al recibir disparos implementando un sistema básico de puntos de vida (HP), además de otorgar puntos al jugador (experiencia).
- **Optimización:** Aunque es un prototipo, se consideraron aspectos básicos de rendimiento, como mantener una tasa de FPS estable para Meta Quest 2 y usar modelos 3D con polígonos reducidos donde fue posible.

7. Conclusiones

El desarrollo del ejemplo "hola-mundo"permitió comprobar que la tecnología seleccionada es adecuada para los objetivos del proyecto. Asimismo, se identificaron los principales retos y requerimientos especificos para las futuras etapas de desarrollo. El trabajo futuro se enfocará en realizar el prototipo de acuerdo al documento y posiblemente expandir sobre su base en un futuro.

Referencias

1. TechTarget. *What is virtual reality? How it's used and how it will evolve*. https://www.techtarget.com/whatis/definition/virtual-reality.
2. Godot Engine Team. *Godot Documentation* (2024). https://docs.godotengine.org/
3. GodotVR community. *Godot XR Tools GitHub Repository* (2023). https://github.com/GodotVR/godot-xr-tools
4. Godot Engine Blog. *Meta funds Godot development for OpenXR and Meta Quest support* (2023). https://godotengine.org/article/meta-funds-openxr/
5. Godot Engine Team. *Godot Asset Library* (2024).https://godotengine.org/asset-library/
6. Unity Technologies. *Unity Documentation* (2024).https://docs.unity3d.com/
7. Epic Games. *Unreal Engine Documentation* (2024).https://docs.unrealengine.com/
8. *Unity unites the indie game industry against its new pricing model*.https://www.theverge.com/23873852/unity-new-pricing-model-news-updates.
9. Godot Engine Community. *Godot Community Resources* (2024). https://godotengine.org/community/.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.